

Data Governance Health Check Report

CONFIDENTIAL: This report contains security posture information. Do not share outside your security or governance team.

Generated: 2026-04-12 15:07 UTC

Role: ACCOUNTADMIN

Coverage: 12 of 12 checks ran (100%)

Scoped databases (Check 1): KOOK_DATA

Data sourced from ACCOUNT_USAGE views, which may have up to 3 hours of latency.

Overall Score: **RED**

The account's data protection posture is critically weak: no PII classification has been run, zero masking or row access policies are applied, and only 1 of 15 user tables carries any governance tag. Identity and access controls are partially in place — the sole remaining user (JBORENSTEINLAWEE) has MFA enabled and is actively logging in, but holds both ACCOUNTADMIN and ORGADMIN directly. Auth and network security is mostly strong: a restrictive network policy (RESTRICTED_ACCESS) limits all connections to a single known IP, though 12 failed logins in 30 days — all from expired OAuth tokens and MFA setup — warrant monitoring.

Top priorities:

- Run SYSTEM\$CLASSIFY on all schemas in KOOK_DATA to discover PII columns and enable data-aware governance
- Apply masking policies to any PII columns discovered by classification

Scorecard

#	Check	Domain	Score
1	PII Classification Coverage	Data Protection	RED
2	Masking & Row Access Policy Coverage	Data Protection	RED
3	Tag Coverage	Data Protection	RED
4	PII Classified but Unmasked	Data Protection	AMBER
5	PII Classified but Untagged	Data Protection	AMBER
6	Privilege Distribution — Overprivileged Roles	Identity & Access	GREEN
7	Humans with Powerful Roles	Identity & Access	AMBER
8	Inactive Users & MFA Disabled	Identity & Access	GREEN
9	Inactive Users with Powerful Roles	Identity & Access	GREEN
10	Failed Login Patterns	Auth & Network	AMBER
11	Network Policy Existence & Application	Auth & Network	GREEN
12	Failed Logins + No Network Policy	Auth & Network	GREEN

Data Protection RED

Check 1: PII Classification Coverage RED

Summary: 0 of 15 tables classified (0%)

Findings: - The schema `SNOWFLAKE.DATA_PRIVACY` does not exist on this account, meaning `SYSTEM$CLASSIFY` has never been run - Scoped database: `KOOK_DATA` (user-selected) - No classification results are available for any table in the account

What this means: Without classification, the account has no automated understanding of which columns contain PII, making it impossible to apply targeted masking or access controls.

Remediation:

```
-- Run classification on each schema in KOOK_DATA
CALL SYSTEM$CLASSIFY('KOOK_DATA.PUBLIC', {'auto_tag': true});

-- Or classify individual tables
CALL SYSTEM$CLASSIFY('KOOK_DATA.PUBLIC.KOOK_ORDERS', {'auto_tag': true});
```

Check 2: Masking & Row Access Policy Coverage **RED**

Summary: 0 columns masked, 0 tables with row access policies

Findings: - Zero masking policies are applied to any column in the account - Zero row access policies are applied to any table in the account - Sensitive data (if any) is fully visible to any role with SELECT access

What this means: All data is returned in plaintext to anyone with table access — there is no column-level or row-level protection in place.

Remediation:

```
-- Step 1: Create a masking policy (example for email)
CREATE OR REPLACE MASKING POLICY email_mask AS (val STRING)
  RETURNS STRING ->
  CASE
    WHEN CURRENT_ROLE() IN ('ACCOUNTADMIN', 'DATA_OWNER') THEN val
    ELSE '***MASKED***'
  END;

-- Step 2: Apply to a column
ALTER TABLE KOOK_DATA.PUBLIC.KOOK_ORDERS
  MODIFY COLUMN EMAIL SET MASKING POLICY email_mask;
```

Check 3: Tag Coverage **RED**

Summary: 1 of 15 user tables tagged (6.7%)

Findings: - Tagged: KOOK_DATA.PUBLIC.KOOK_ORDERS — tag: SENSITIVITY = PII - 14 tables have no governance tags at all - Total user tables across account: 15

What this means: Only one table has been classified with a governance tag. Without tags, tag-based masking policies cannot be applied and data discovery is severely limited.

Remediation:

```
-- Create a governance tag (if not already created)
CREATE TAG IF NOT EXISTS KOOK_DATA.PUBLIC.SENSITIVITY
  ALLOWED_VALUES 'PII', 'SENSITIVE', 'PUBLIC', 'INTERNAL';

-- Apply tags to untagged tables (repeat for each table)
ALTER TABLE <database>.<schema>.<table>
  SET TAG KOOK_DATA.PUBLIC.SENSITIVITY = 'PII';

-- Or run auto-tagging via classification
CALL SYSTEM$CLASSIFY('KOOK_DATA.PUBLIC', {'auto_tag': true});
```

Check 4: PII Classified but Unmasked **AMBER**

Summary: Unable to evaluate — no classification data available

Findings: - SNOWFLAKE.DATA_PRIVACY.CLASSIFICATION_RESULT does not exist on this account - Cannot determine which PII columns lack masking without classification results - This check will become actionable after running SYSTEM\$CLASSIFY

What this means: This check cannot produce a definitive result because the prerequisite (PII classification) has not been run. The score is AMBER rather than GREEN because the absence of classification data means unmasked PII may exist undetected.

Remediation:

```
-- Run classification first, then re-run this health check
CALL SYSTEM$CLASSIFY('KOOK_DATA.PUBLIC', {'auto_tag': true});
```

Check 5: PII Classified but Untagged **AMBER**

Summary: Unable to evaluate — no classification data available

Findings: - SNOWFLAKE.DATA_PRIVACY.CLASSIFICATION_RESULT does not exist on this account - Cannot determine which PII columns lack tags without classification results - This check will become actionable after running SYSTEM\$CLASSIFY

What this means: Without classification data, we cannot identify PII columns that should be tagged but are not. The AMBER score reflects the risk that untagged PII may exist.

Remediation:

```
-- Run classification with auto_tag enabled to both classify and tag
CALL SYSTEM$CLASSIFY('KOOK_DATA.PUBLIC', {'auto_tag': true});
```

Identity and Access **AMBER**

Check 6: Privilege Distribution — Overprivileged Roles **GREEN**

Summary: 0 custom roles inherit admin-level privileges directly

Findings: - No custom roles were found that inherit from ACCOUNTADMIN, SECURITYADMIN, SYSADMIN, or ORGADMIN - USERADMIN holds CREATE USER and CREATE ROLE privileges — this is a Snowflake default and expected - SHOPIFY_CONNECTOR holds CREATE DATABASE — granted by the application package

What this means: The role hierarchy follows Snowflake best practices with no unexpected privilege escalation paths through custom roles.

Remediation: No action required.

Check 7: Humans with Powerful Roles **AMBER**

Summary: 1 human user holds ACCOUNTADMIN + ORGADMIN

Findings: - JBORENSTEINLAWEE — Likely Human (password-based auth, no RSA key) — holds ACCOUNTADMIN and ORGADMIN - This is the only user on the account

What this means: A single human user holds the two most powerful roles in the account. While expected for a single-user account, this concentrates all risk. Best practice is to limit ACCOUNTADMIN to break-glass scenarios and use lower-privilege roles for daily work.

Remediation:

```
-- Create a lower-privilege role for daily operations
CREATE ROLE IF NOT EXISTS DATA_ENGINEER;
GRANT ROLE DATA_ENGINEER TO USER JBORENSTEINLAWEE;

-- Use ACCOUNTADMIN only when necessary
-- Consider setting up a dedicated admin user for break-glass access
```

Check 8: Inactive Users & MFA Disabled **GREEN**

Summary: 0 of 1 users are inactive or missing MFA

Findings: - 1 total user on the account (JBORENSTEINLAWEE) - No inactive users found (all have logged in within 90 days) - No users without MFA found - Note: TEST_PUBLIC_USER from a previous run appears to have been removed

What this means: All users on the account are active and have MFA enabled. This is the ideal state.

Remediation: No action required.

Check 9: Inactive Users with Powerful Roles **GREEN**

Summary: 0 admin users are inactive or missing MFA

Findings: - No users with ACCOUNTADMIN, SECURITYADMIN, SYSADMIN, or ORGADMIN are inactive - The only admin user (JBORENSTEINLAWEE) has MFA enabled and is actively logging in

What this means: Admin-level access is not exposed through dormant or poorly secured accounts.

Remediation: No action required.

Auth and Network **AMBER**

Check 10: Failed Login Patterns **AMBER**

Summary: 12 failed logins out of 877 total (1.4%) in the last 30 days

Findings: - 8 failures: OAUTH_ACCESS_TOKEN_EXPIRED — JBORENSTEINLAWEE from 24.148.69.158 (Apr 6–12) — expired OAuth tokens from IDE/connector sessions - 3 failures: TOTP_INVALID_CODE_REMAINING_ATTEMPTS — JBORENSTEINLAWEE from 24.148.69.158 (Apr 6) — MFA code mistyped - 1 failure: SECOND_FACTOR_AUTHENTICATION_REQUIRED —

JBORENSTEINLAWEE from 24.148.69.158 (Apr 6) — MFA prompt before enrollment completed - Pattern flag: same user (JBORENSTEINLAWEE) has 8+ failures from expired OAuth tokens - All failures originate from the same known IP (24.148.69.158) — no external threat indicators

What this means: The failure count exceeds 10, triggering the AMBER threshold. All failures are attributable to normal operational causes (expired tokens, MFA setup) from the account's sole user at the known IP. No brute-force or credential-stuffing patterns detected.

Remediation:

```
-- Consider rotating OAuth client credentials or extending token TTL
-- to reduce expired-token failures
-- Monitor for sustained spikes as the account grows
```

Check 11: Network Policy Existence & Application GREEN

Summary: 2 network policies exist; RESTRICTED_ACCESS applied at account level

Findings: - RESTRICTED_ACCESS — created 2026-04-11 — allowed IP: 24.148.69.158 — applied at account level - ALLOW_ALL_IPS — created 2026-04-05 — allowed IP: 0.0.0.0/0 — not applied (superseded) - The account-level policy restricts all connections to a single known IP address

What this means: The account is protected by a restrictive network policy that limits access to one specific IP. This is a strong security control that prevents unauthorized network access even if credentials are compromised.

Remediation:

```
-- Consider removing the unused ALLOW_ALL_IPS policy to reduce confusion
DROP NETWORK POLICY ALLOW_ALL_IPS;

-- If additional IPs are needed, update the active policy
ALTER NETWORK POLICY RESTRICTED_ACCESS
SET ALLOWED_IP_LIST = ('24.148.69.158', '<additional_ip>');
```

Check 12: Failed Logins + No Network Policy GREEN

Summary: No cross-reference risk — account has a restrictive network policy

Findings: - The account-level network policy (RESTRICTED_ACCESS) limits access to a single IP - Failed logins are moderate-volume but non-suspicious (all from known IP) - No users are both failing logins and unprotected by network restrictions

What this means: The combination of a restrictive network policy and operationally-caused failed logins means the account is not exposed to the credential-stuffing + open-network risk pattern.

Remediation: No action required.

Prioritized Recommendations

Ranked by risk (Red items first, then Amber):

1. **RED Run PII classification** — Execute `SYSTEM$CLASSIFY` on all schemas in `KOOK_DATA` to discover sensitive columns. This unblocks Checks 1, 4, and 5 and enables data-aware governance.
2. **RED Apply masking policies** — After classification identifies PII columns, create and apply masking policies so sensitive data is not visible in plaintext to all roles.
3. **RED Expand tag coverage** — Tag all 15 user tables with governance tags (e.g., `SENSITIVITY`). Use `auto_tag: true` with `SYSTEM$CLASSIFY` to automate this, or apply tags manually.
4. **AMBER Reduce daily use of ACCOUNTADMIN** — Create a lower-privilege role for routine data engineering work. Reserve `ACCOUNTADMIN` for administrative tasks only.
5. **AMBER Monitor OAuth token expiry** — 8 of 12 failed logins are expired OAuth tokens. Consider extending token TTL or refreshing tokens proactively in your connectors.
6. **AMBER Remove unused ALLOW_ALL_IPS policy** — The `ALLOW_ALL_IPS` network policy is no longer applied but still exists. Drop it to eliminate confusion and reduce the risk of accidental reapplication.